



UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA CAMPUS VALLE LAS PALMAS

PROYECTO IOT

OXÍMETRO

ASIGNATURA: ORGANIZACIÓN DE COMPUTADORAS

DOCENTE: DR. JOSÉ MANUEL ROCHA

ALUMNO: ARATH REYES GONZALEZ

MAYO 2026



ÍNDICE

- ▶ OBJETIVO
- ▶ INTRODUCCIÓN
- ▶ COMPONENTES
- ▶ DESCRIPCIÓN DEL MECANISMO OPERATIVO
- ▶ ENSAMBLAJE
- ▶ DIAGRAMA DE CIRCUITOS
- ▶ CÓDIGO
- ▶ CONCLUSIÓN
- ▶ BIBLIOGRÁFICAS

OBJETIVO

Diseñar, implementar y validar un sistema embebido portátil de monitoreo cardiaco y saturación de oxígeno. Mediante la creación de un proyecto IoT. [5]



INTRODUCCIÓN

Tradicionalmente, el monitoreo de signos vitales se limitaba al entorno clínico por el uso de equipos costosos y personal especializado. Actualmente, el avance tecnológico ha permitido desarrollar dispositivos médicos accesibles basados en IoT. [5]

¿QUÉ ES UN OXÍMETRO?

El oxímetro de pulso es un dispositivo que mide el nivel de oxígeno en la sangre de forma no invasiva. Funciona mediante haces de luz que atraviesan el dedo, permitiendo estimar el porcentaje de oxígeno presente en los glóbulos rojos. [1]



FUNCIONAMIENTO DEL DISPOSITIVO

El oxímetro de pulso utiliza espectrofotometría y fotopletismografía. Emite luz roja e infrarroja a través del tejido, analiza su absorción por la hemoglobina oxigenada y desoxigenada, y calcula la saturación de oxígeno (SpO_2) y la frecuencia del pulso. [1]

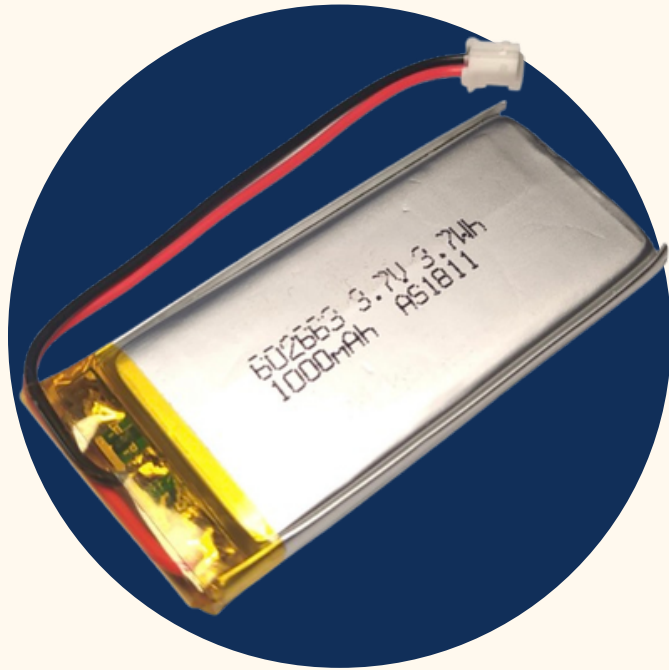


Imagen de referencia visual de como funciona un oxímetro tradicional

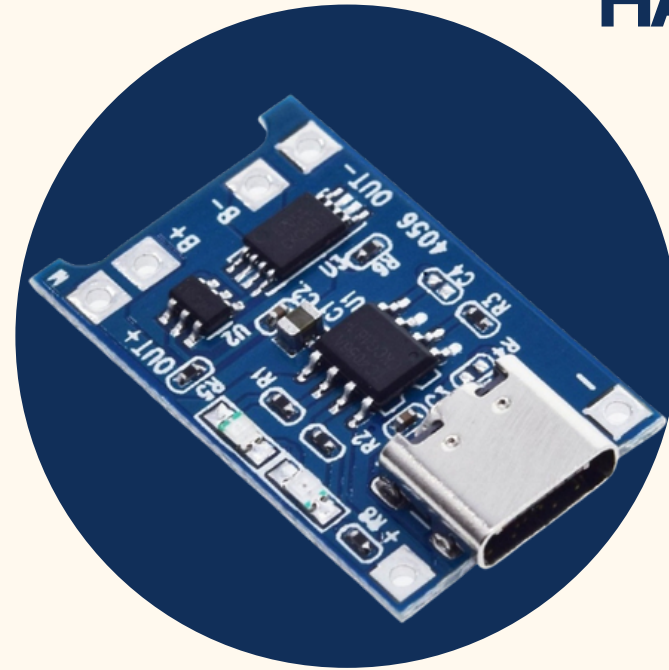
(1) "Oxímetro de pulso: MedlinePlus enciclopedia médica ilustración." https://medlineplus.gov/spanish/ency/esp_imagepages/20005.htm

COMPONENTES

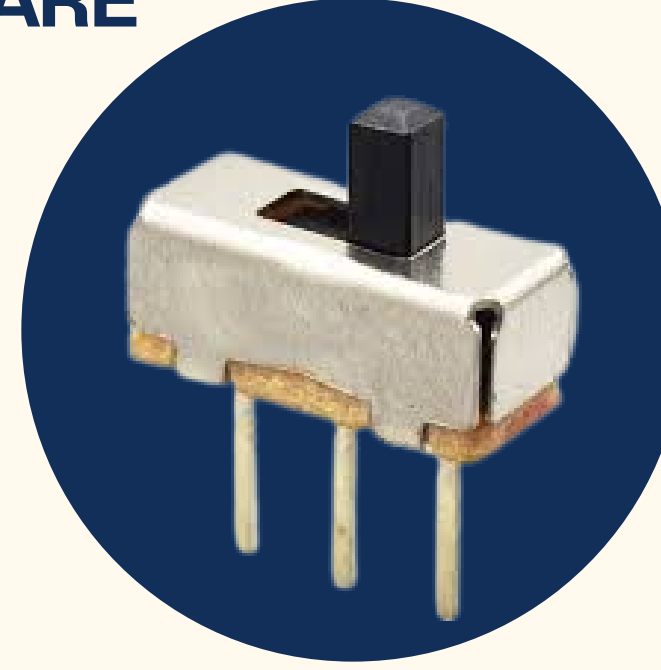
HARDWARE



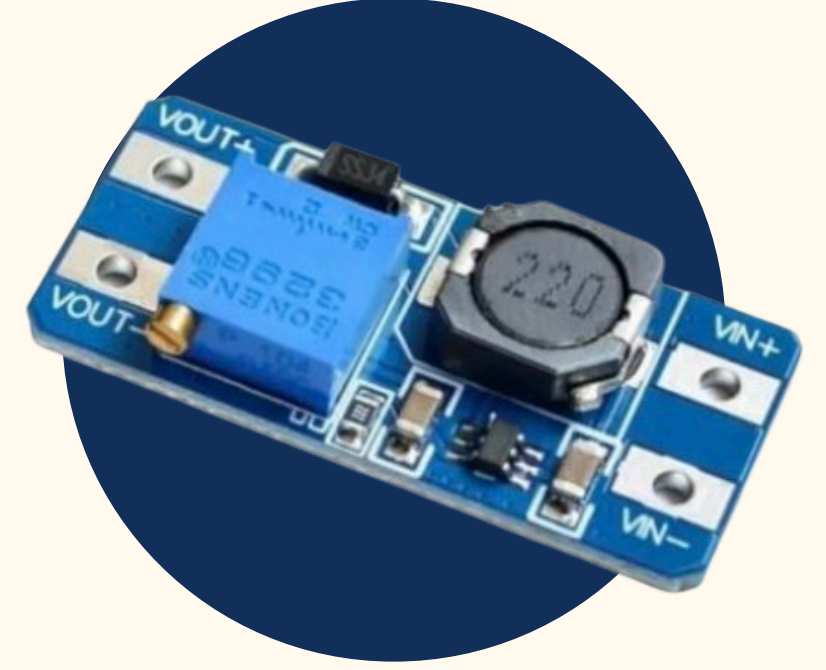
Batería LiPo3.7



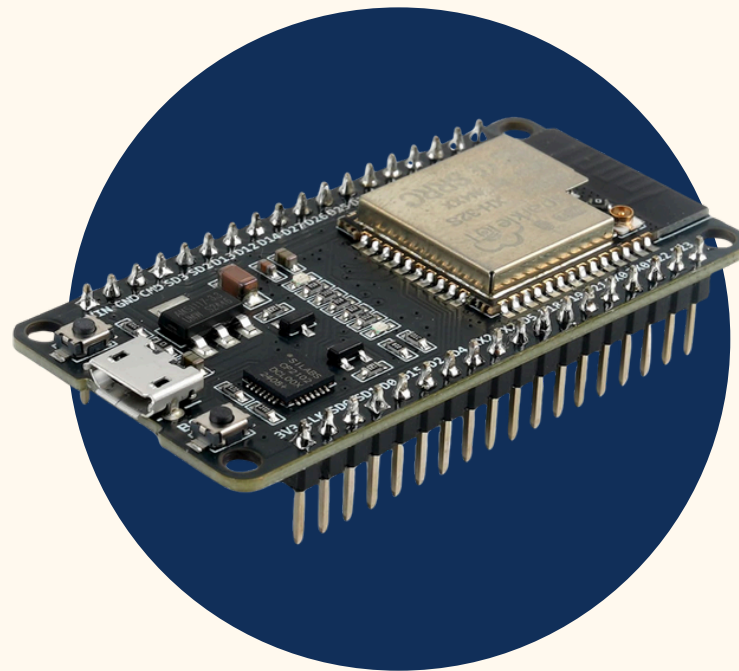
Tp4056



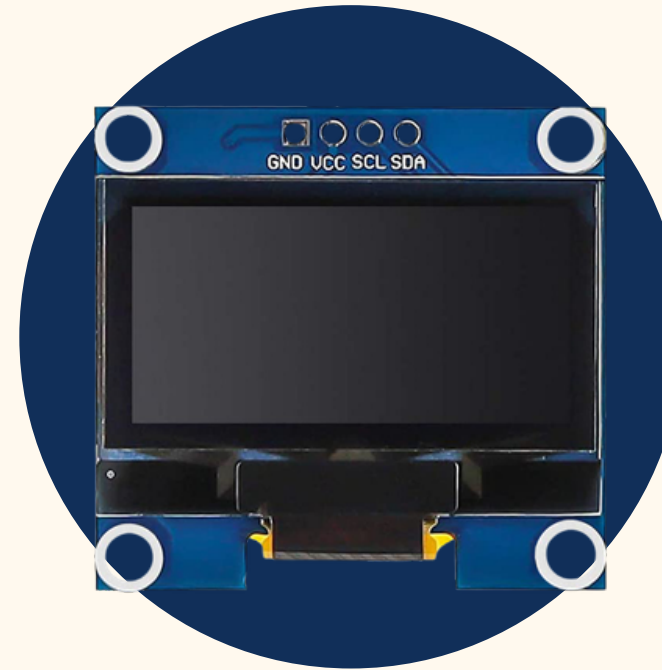
Switch



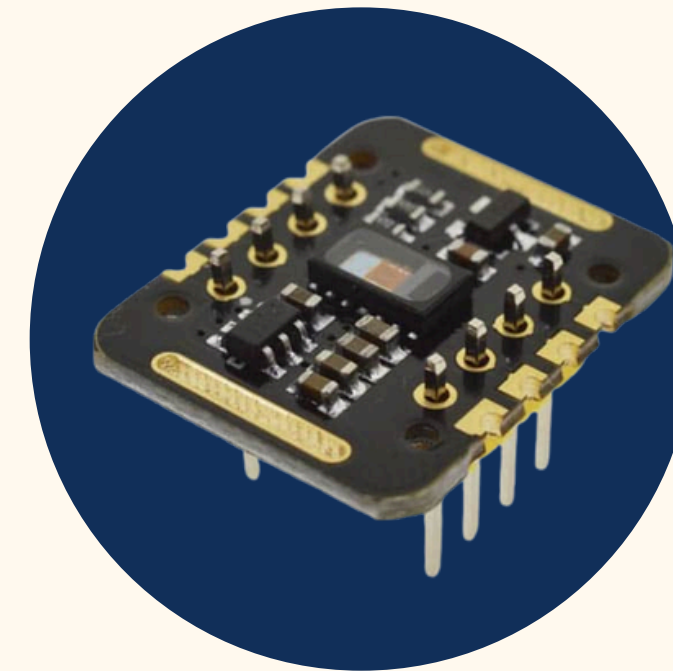
Mt3608 boost



Esp32



Oled SSD1306



Max30102

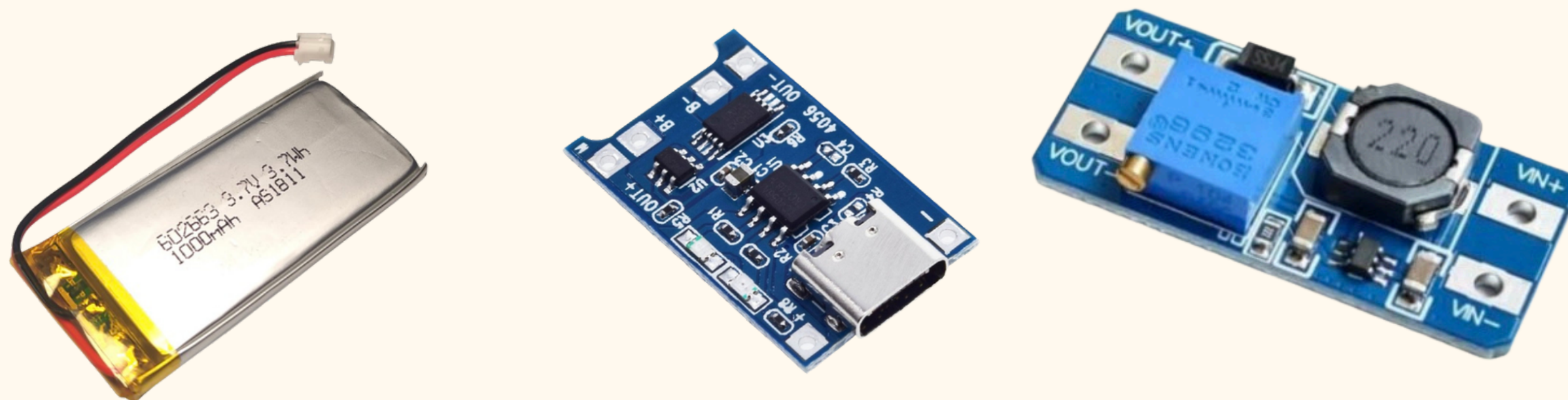
EXPLICACIÓN COMPONENTES

- ▶ **Batería LiPo36:** Fuente de alimentación portátil del sistema. Proporciona energía al dispositivo.
- ▶ **TP4056:** Módulo cargador que regula la carga segura de la batería LiPo mediante su entrada USC-C.
- ▶ **Switch:** Interruptor que permite encender y apagar el circuito controlando el paso de corriente.
- ▶ **MT3608 Boost:** Conventidor elevador que aumenta el voltaje de la batería 5V para alimentar el ESP32.
- ▶ **ESP32:** Microcontrolador principal que procesa datos, controla sensores, visualizar información y transmite datos por WiFi.
- ▶ **OLED SSD1306:** Pantalla de bajo consumo que muestra BPM, SpO2 y estado del sistema en tiempo real.
- ▶ **MAX30102:** Sensor óptico que mide frecuencia cardíaca y saturación de oxígeno mediante fotoplestismografía. [2]

ENSAMBLAJE

1. INTEGRACIÓN DEL SISTEMA DE ALIMENTACIÓN

El ensamblaje inició con el sistema de alimentación portátil. Se conectó una batería LiPo de 3.7 V al módulo TP4056, encargado de gestionar la carga segura mediante entrada USB-C. Debido a que el voltaje de la batería no era suficiente para alimentar correctamente el ESP32, se incorporó un convertidor elevador MT3608, ajustado para entregar una salida estable de 5 V. Además, se añadió un interruptor (switch SPDT) para controlar el encendido y apagado del dispositivo sin desconectar la batería. [5]



ENSAMBLAJE

2. CONEXIÓN A MICROCONTROLADOR Y PERIFÉRICOS

Posteriormente, el ESP32 se integró como unidad central del sistema. Este microcontrolador recibe la alimentación del circuito y coordina el funcionamiento de todos los componentes.

La pantalla OLED SSD1306 y el sensor MAX30102 se conectaron al ESP32 mediante el protocolo I2C, utilizando los pines GPIO21 (SDA) y GPIO22 (SCL). Esta configuración permitió que ambos dispositivos compartieran el mismo canal de comunicación, reduciendo el cableado y simplificando el ensamblaje.

ENSAMBLAJE

¿QUÉ ES EL PROTOCOLO I2C?

El bus I2C (Inter-Integrated Circuit) es un protocolo serie sincrónico de dos hilos: SDA (datos) y SCL (reloj)

La elección de I2C sobre SPI se justifica por dos razones: reduce a dos las líneas de señal necesarias independientemente del número de dispositivos, simplificando el cableado en la perfboard; y el ESP32 asigna GPIO21/GPIO22 al bus I2C o por defecto, eliminando configuración adicional.

Analogía del protocolo I2C:

"Imagina que el ESP32 es el maestro de una clase y los dispositivos (OLED, sensor) son los alumnos. El maestro solo necesita 2 cosas para comunicarse con todos: un micrófono (SCL) que marca el ritmo de cuando hablar, y un canal de radio compartido (SDA) por donde todos envían y reciben mensajes.

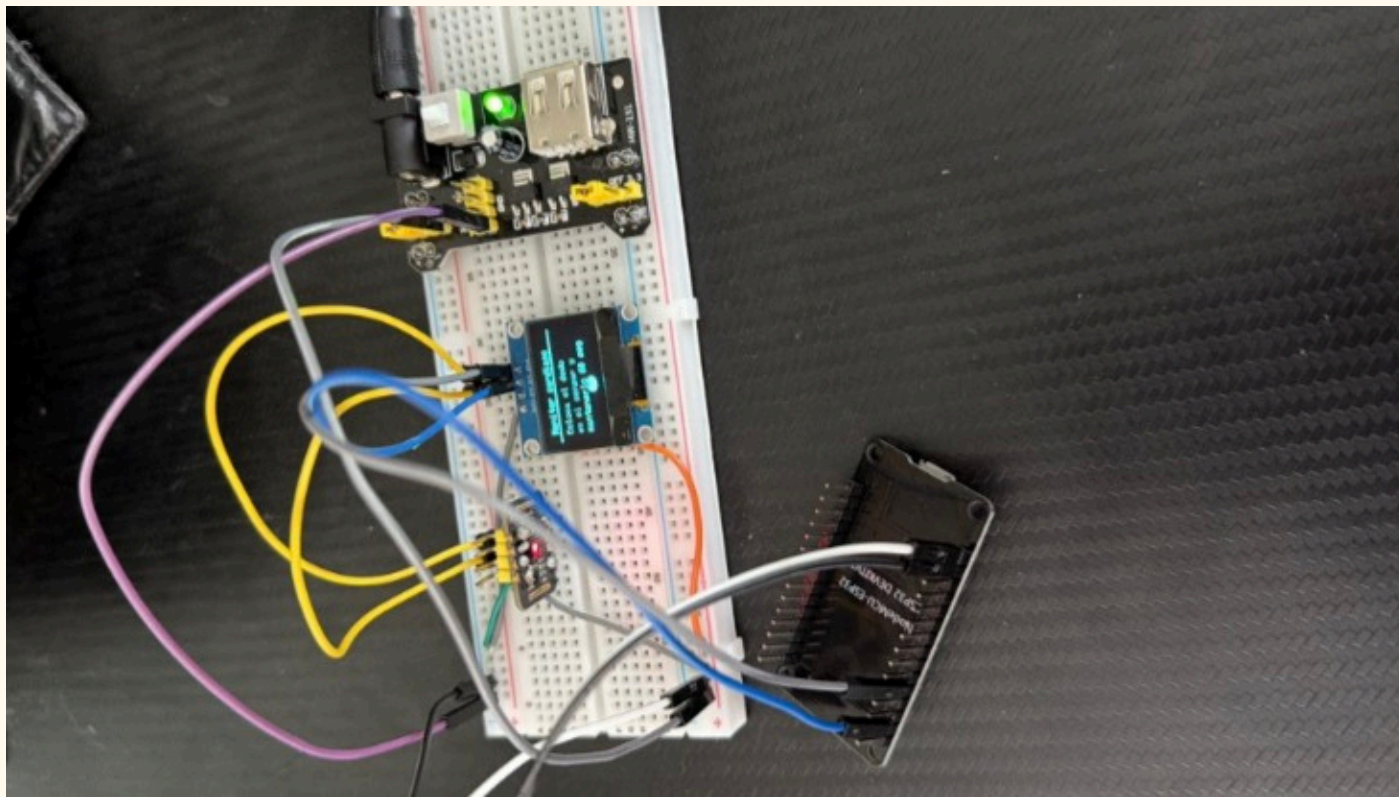
Cada alumno tiene un número de lista (dirección I2C) y solo responde cuando el maestro lo llama por ese número."

ENSAMBLAJE

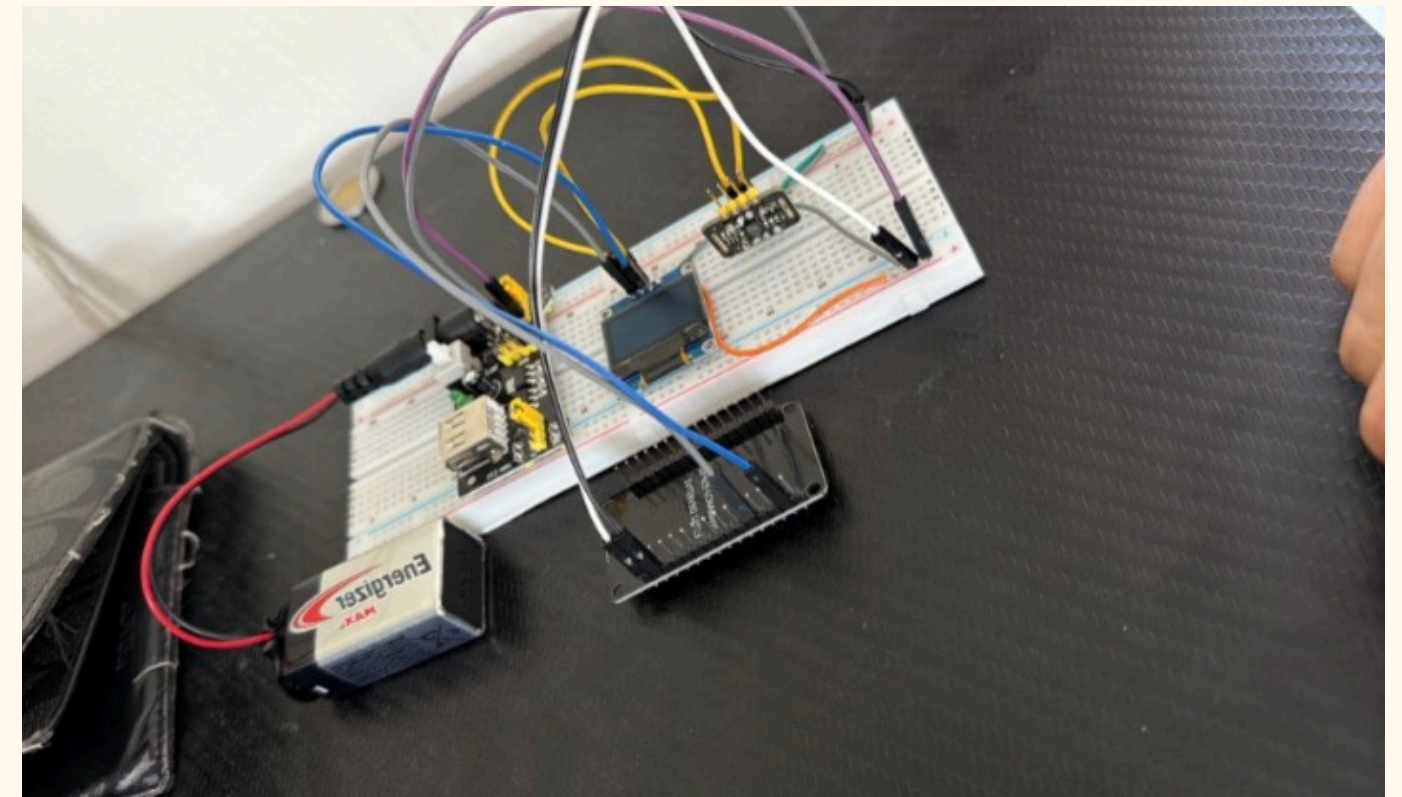
3.PROTOTIPADO Y VALIDACIÓN INICIAL

Antes del montaje definitivo, el circuito se ensambló inicialmente sobre una protoboard.

Esta etapa permitió verificar conexiones, alimentación, comunicación entre módulos y funcionamiento general del sistema antes de realizar soldaduras permanentes.



Primera conexion en protoboard.



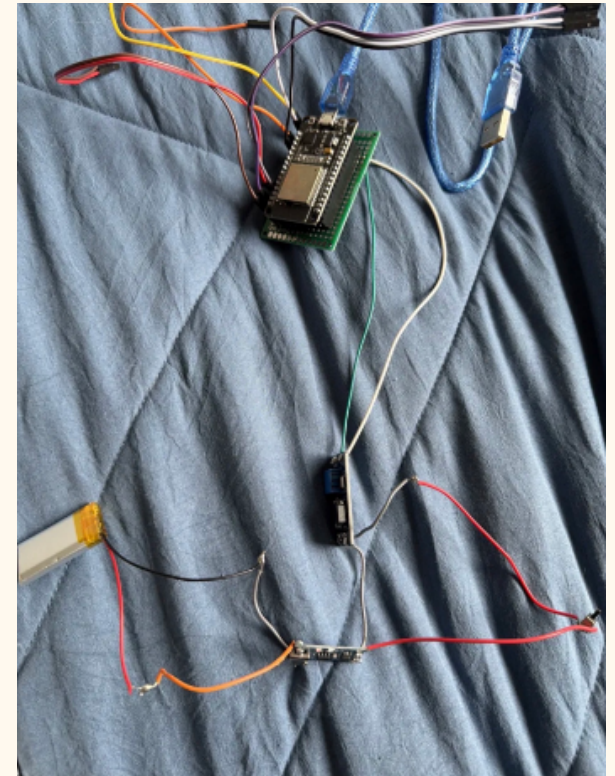
Verificación del funcionamiento inicial.

ENSAMBLAJE

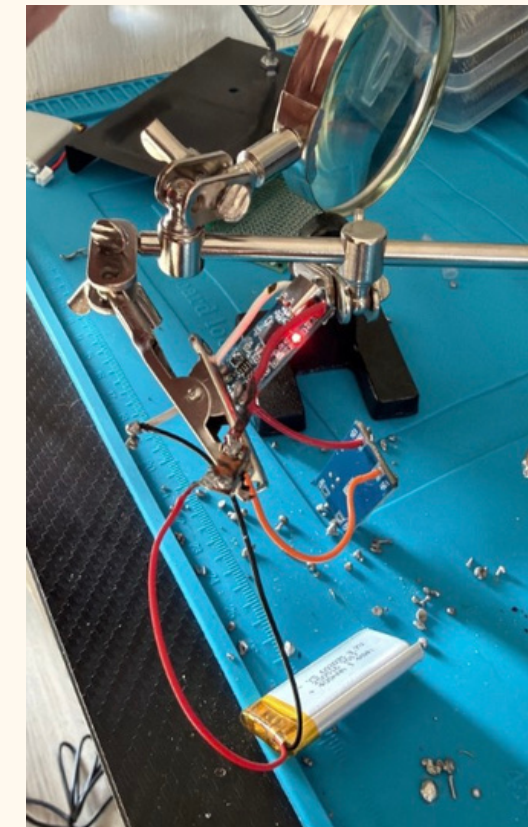
4. ENSAMBLAJE FINAL Y SOLDADURA

Una vez comprobado el funcionamiento del prototipo, los componentes fueron transferidos a una perfboard, donde se realizó la soldadura definitiva para mejorar la estabilidad mecánica y eléctrica del dispositivo.

Durante esta etapa se corrigieron problemas relacionados con soldaduras frías, cortocircuitos y distribución del voltaje, hasta obtener una versión portátil completamente funcional.

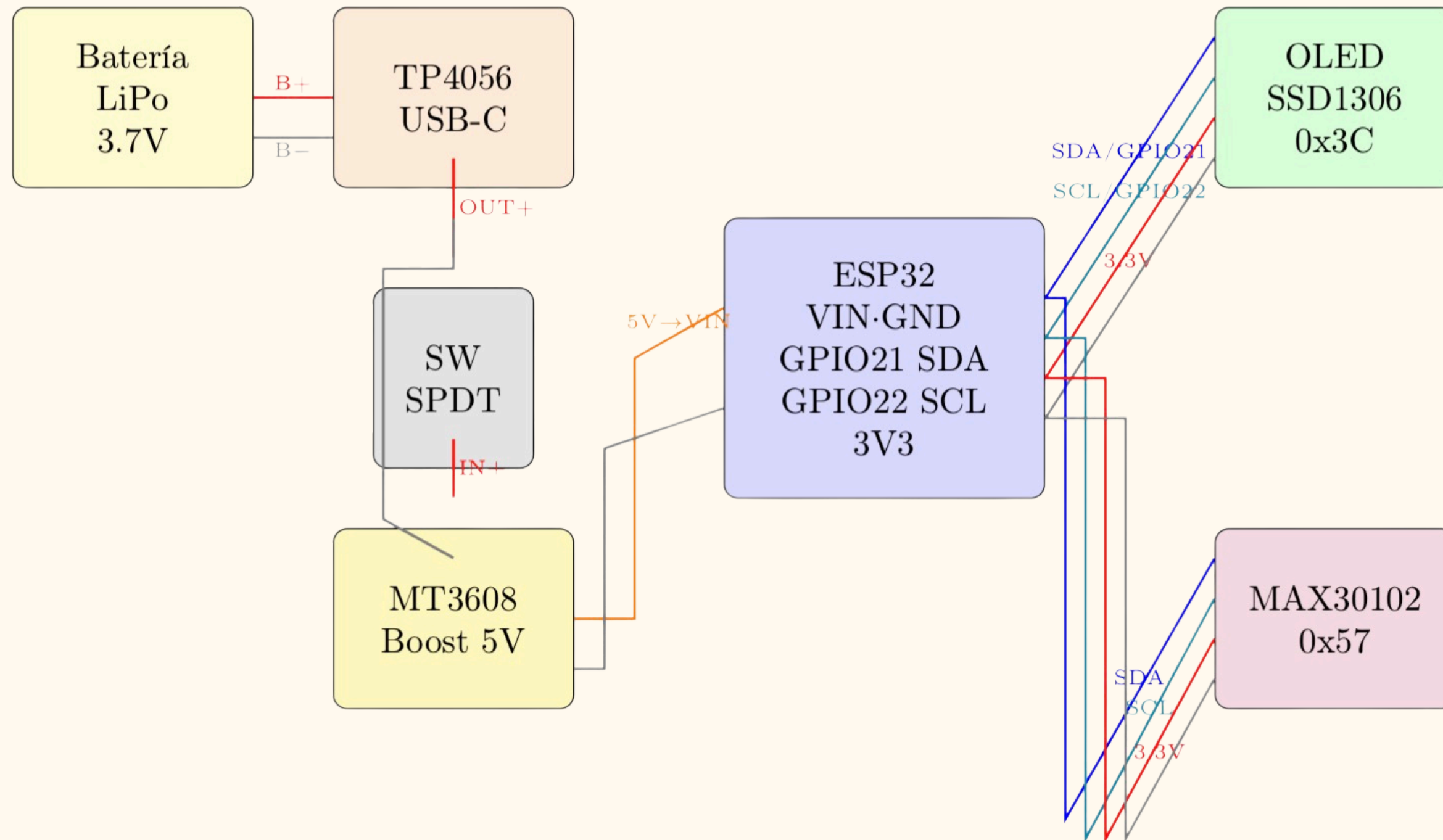


Detalles del ensamblado final.



Proceso de soldadura.

DIAGRAMA DE CIRCUITOS



CÓDIGO

El código consiste en 4 etapas distintas:

- ① Las librerías necesarias
- ② Maquinas de estado
- ③ Servidor Web
- ④ JSON

CÓDIGO

Las librerías son bloques de código ya escrito por otras personas para no tener que crear algoritmos desde 0, en arduino son necesario agregarlos con el `#include` al inicio del código. Las librerías necesarias para este proyecto fueron las siguientes: [5]

1. ***Wire.h***:

Es la que permite la comunicación I2C entre el ESP32 y los 2 dispositivos, así que sin ella no podría haber comunicación.

2. ***Wifi.h*** y ***WebServer.h***

Estas 2 librerías permiten que el ESP32 pueda crear su red wifi el cual funcione como un servidor web.

3. ***MAX30105.H*** - ***heartrate.h*** - ***spo2_algorithm.h***

Son 3 librerías del fabricante del sensor. La primera controla el hardware del sensor, la segunda detecta los latidos del corazón y la tercera calcula el porcentaje de la oxigenación en la sangre.

4. ***_GFX.h*** y ***Adafruit_SSD1306.h***

Controlan la pantalla OLED, la primera provee funciones de dibujo y la segunda es el driver del modelo en específico.

5. ***ArduinoJson.h***

Permite crear y leer archivos JSON, que es el formato usado para enviar datos a través del sensor al navegador web

CÓDIGO

Maquinas de estados:

La maquina de estados es el corazon del programa. Existen 4 estados, en lugar de ejecutar el codigo de forma lineal de arriba hacia abajo, el ESP32 siempre se encuentra en uno de los 4 estados y a cada uno le corresponden distintas tareas. Esto se logra con el switch case dentro del loop, que revisa en que estado corresponde estar.

Estado 0: Esperando

El ESP32 se encuentra en un modo reposo mostrando el mensaje de inicial para empezar la medicion, hasta no detectar un dedo mediante el infrarrojo se mantiene en este estado.

Estado 1: Midiendo

El ESP32 mide durante 60 segundos, en cada ciclo calcula el BPM y SPO2, refrescando resultados en el servidor y oled.

Estado 2: Mostrar resultados

Es la encargada de mostrar los resultados una vez el estado 1 haya acabado.

Estado 3: Subiendo

Es la que se encarga que con el access point, muestre el IP donde se puedan ver los resultados, una vez acabando este estado se regresa a esperando y así continua el ciclo.

CÓDIGO

Servidor Web:

En esta fase el ESP no mide, sino que funciona como el access point para usuarios, creando una red y asignando una IP a esta donde se puedan ver los resultados.

La librería encargada de esta siendo WebServer.

JSON (Como puente):JavaScript Object Notation

Es un formato de texto simple para poder organizar y transportar datos. Es el estandar mas utilizado en internet para que 2 sistemas intercambien información. [6]

Se utiliza JSON en vez de HTML, ya que la pagina se envía solo una vez cuando el usuario la abre. Si cada vez que se calculase el BPM o SPO2 se enviara la pagina, seria lento. En cambio, cada 1.5 segundos el JavaScript pide solo los datos y actualiza los datos constantemente en vez de recargar todo por completo.

CONCLUSIÓN

En conclusión, este proyecto evidencia como los avances en ingeniería han permitido desarrollar dispositivos médicos accesibles y funcionales en cuanto al monitoreo de signos vitales.

Dispositivos como el oxímetro de pulso, aunque aparentemente simples, permiten evaluar parámetros esenciales para la salud y apoyar la detección y seguimiento de alteraciones clínicas en millones de personas.

Esto nos demuestra el impacto que la tecnología puede tener en la medicina y en el acceso al cuidado de la salud.

Esto da como resultado una armonía entre proyectos IoT y la medicina. Ya que cualquier persona podría crear proyectos relacionados con la medicina y poder obtener respuestas a padecimientos con la ayuda tecnológica.

Referencias:

[1] "Oxímetro de pulso: MedlinePlus enciclopedia médica ilustración." https://medlineplus.gov/spanish/ency/esp_imagepages/20005.htm

[2] L. Llamas, "Pulsímetro y oxímetro con Arduino y MAX30102," Luis Llamas, Apr. 12, 2020. <https://www.luisllamas.es/en/pulse-oximeter-arduino-max30102>

[3]<https://protonestiot.medium.com/diy-oximeter-with-max30102-and-esp32-52ec3ffb2446/>

[4] L. Del Valle Hernández, "SSD1306 pantalla OLED con Arduino y ESP8266 I2C," Programarfacil Arduino y Home Assistant, Jan. 13, 2022. <https://programarfacil.com/blog/arduino-blog/ssd1306-pantalla-oled-con-arduino/>

[5]Seyer, "Proyecto-Final-OC/Proyecto_Final-1.pdf at main · Seyer24/Proyecto-Final-OC," GitHub. https://github.com/Seyer24/Proyecto-Final-OC/blob/main/Proyecto_Final-1.pdf

[6]"JSON." <https://www.json.org/json-es.html>

[7]

[8]